

Table of Contents

1. [Project Overview](#)
2. [Prerequisites & Environment Setup](#)
3. [Phase 1 — Frontend \(React + Vite + TailwindCSS\)](#)
4. [Phase 2 — Backend \(Spring Boot + Kafka Producer\)](#)
5. [Phase 3 — Calculator Lambda \(MSK Consumer\)](#)
6. [Phase 4 — Infrastructure as Code \(AWS CDK\)](#)
7. [Phase 5 — Local Development with Docker Compose](#)
8. [Phase 6 — Build & Deploy to AWS](#)
9. [Phase 7 — Migration: ECS Fargate → API Gateway Lambda](#)
10. [Phase 8 — Migration: DynamoDB Polling → WebSocket Push](#)
11. [Phase 9 — Frontend Update for WebSocket](#)
12. [Phase 10 — Deployment Issues & Fixes](#)
13. [Architecture Evolution Summary](#)
14. [Lessons Learned](#)

1. Project Overview

What We're Building

A **compound interest calculator** that demonstrates a production-grade, event-driven, serverless architecture on AWS. The user enters investment parameters and receives the calculated future value.

The Compound Interest Formula

A = P × (1 + r/n)^(n × t)

Variable	Description
A	Final amount
P	Principal (initial investment)
r	Annual interest rate (as decimal)
n	Compounding frequency per year
t	Time in years

Technology Stack

Layer	Technology
Frontend	React 18, Vite, TailwindCSS, Lucide Icons
Backend (v1)	Java 17, Spring Boot 3.2, ECS Fargate
Backend (v2)	Java 17, AWS Lambda (API Handler)
Compute	Java 17, AWS Lambda (Calculator)
Messaging	Apache Kafka via Amazon MSK
Database (v1)	Amazon DynamoDB
Database (v2)	Removed (WebSocket push)
Infrastructure	AWS CDK (TypeScript)
Hosting	Amazon S3 + CloudFront

Architecture Versions

The project evolved through **three architecture versions**:

- 1. **V1**: React → CloudFront → ECS Fargate (Spring Boot) → Kafka → Lambda → DynamoDB → Poll
- 2. **V2**: React → CloudFront → API Gateway (HTTP) → Lambda API → Kafka → Lambda → DynamoDB → Poll
- 3. **V3**: React → CloudFront + WebSocket API Gateway → Lambda API → Kafka → Lambda → WebSocket Push

2. Prerequisites & Environment Setup

Required Software

Install the following before starting:

```
# Node.js 18+ (via nvm recommended)
nvm install 20
nvm use 20

# Java 17 (via SDKMAN recommended)
sdk install java 17.0.9-amzn

# Maven 3.9+
sdk install maven 3.8.7

# AWS CLI v2
brew install awscli

# AWS CDK
npm install -g aws-cdk

# Docker (for local dev and building images)
brew install --cask docker
```

AWS Configuration

```
aws configure
# AWS Access Key ID: <your-key>
# AWS Secret Access Key: <your-secret>
# Default region: us-east-1
# Default output format: json
```

Project Initialization

```
mkdir calculation-using-kafka-backend
cd calculation-using-kafka-backend
git init
```

3. Phase 1 — Frontend (React + Vite + TailwindCSS)

Step 1: Create the React Project

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
```

Step 2: Install Dependencies

```
npm install lucide-react axios
npm install -D tailwindcss @tailwindcss/vite
```

Step 3: Configure TailwindCSS

Update `vite.config.js` to include the TailwindCSS plugin and configure `tailwind.config.js` with content paths.

Step 4: Create the API Service (`src/api/calculationService.js`)

The API service handles communication with the backend:

- **V1 (REST + Polling)**: Uses Axios to `POST /api/calculations` then polls `GET /api/calculations/{id}` every second until `status === 'COMPLETED'`.
- **V3 (WebSocket)**: Opens a persistent WebSocket connection and sends/receives JSON messages.

Step 5: Create Components

`CalculatorForm.jsx`

- Collects **principal**, **annualRate**, **years**, **compoundingFrequency** from user input
- Validates inputs before submission
- Calls `onSubmit(formData)` callback

`ResultDisplay.jsx`

- Receives the `result` prop with all calculation details
- Renders:
 - Input parameters summary
 - Final amount with formatting
 - Growth chart (visual bar chart)
 - Year-by-year breakdown table

`App.jsx`

- Manages state: `isLoading`, `status`, `result`, `error`
- Handles form submission and result display
- Shows connection status and processing indicators
- Contains "How It Works" section and tech badges

Step 6: Build

```
npm run build    # Outputs to dist/
```

4. Phase 2 — Backend (Spring Boot + Kafka Producer)

Step 1: Initialize the Spring Boot Project

Create `backend/pom.xml` with:

- `spring-boot-starter-web`
- `spring-kafka`
- `aws-java-sdk-dynamodb` (for DynamoDB operations)

Step 2: Create the Data Models

`CalculationRequest.java`

```
public class CalculationRequest {  
    private double principal;  
    private double annualRate;  
    private int years;  
    private int compoundingFrequency;  
}
```

`CalculationResponse.java`

Response DTO including `calculationId`, `status`, all inputs, `finalAmount`, and timestamps.

`CalculationEvent.java`

Kafka event model with all calculation inputs plus `calculationId`.

Step 3: Create Services

`KafkaProducerService.java`

- Injects Spring's `KafkaTemplate<String, String>`
- Serializes `CalculationEvent` to JSON
- Publishes to the `calculation-requests` topic with `calculationId` as the key

`DynamoDbService.java`

- Uses AWS SDK's `DynamoDbClient`
- `putCalculation()` : Writes a PENDING record with all inputs
- `getCalculation()` : Reads a calculation by ID
- `updateCalculation()` : Updates status and result fields

`CalculationService.java`

- Orchestrates the flow:
 1. Generate UUID for `calculationId`
 2. Save PENDING record to DynamoDB
 3. Publish event to Kafka
 4. Return response with `calculationId`

Step 4: Create the REST Controller

CalculationController.java

```
@PostMapping("/api/calculations")
public ResponseEntity<CalculationResponse> submitCalculation(@RequestBody CalculationRequest request);

@GetMapping("/api/calculations/{id}")
public ResponseEntity<CalculationResponse> getCalculation(@PathVariable String id);

@GetMapping("/api/calculations/health")
public ResponseEntity<String> health();
```

Step 5: Configure Kafka and DynamoDB

KafkaConfig.java

- Configure producer with `StringSerializer` for keys and values
- Set bootstrap servers from environment variable
- Configure acknowledgment modes

DynamoDbConfig.java

- Create `DynamoDbClient` bean
- Configure region and table name from environment

Step 6: Dockerfile

```
FROM amazoncorretto:17-alpine
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Step 7: Build

```
cd backend
mvn clean package -DskipTests
docker build -t compound-calc-backend .
```


5. Phase 3 — Calculator Lambda (MSK Consumer)

Step 1: Create the Maven Project

Create `lambda-calculator/pom.xml` with:

- `aws-lambda-java-core`
- `aws-lambda-java-events`
- `aws-lambda-java-log4j2`
- `kafka-clients`
- `jackson-databind`
- `aws-java-sdk-dynamodb` (V1, later replaced)

Use the `maven-shade-plugin` to create an uber-JAR.

Step 2: Create the Event Model

`CalculationEvent.java`

```
public class CalculationEvent {
    private String calculationId;
    private double principal;
    private double annualRate;
    private int years;
    private int compoundingFrequency;
    private String connectionId; // Added in V3
    private String wsCallbackUrl; // Added in V3
}
```

Step 3: Create the Handler

`CompoundInterestHandler.java`

The Lambda handler implements `RequestHandler<KafkaEvent, Void>` :

1. **Receive:** MSK event source mapping delivers Kafka records in batches
2. **Deserialize:** Parse each record's value as `CalculationEvent`
3. **Compute:** Apply the compound interest formula
4. **Return result:**
 - **V1/V2:** Write result to DynamoDB (`status: COMPLETED` , `finalAmount`)
 - **V3:** Push result via WebSocket Management API (`PostToConnection`)

Compound Interest Calculation

```
double r = annualRate / 100.0;
double amount = principal * Math.pow(1 + r / n, n * years);
double finalAmount = Math.round(amount * 100.0) / 100.0;
```

Step 4: Build

```
cd lambda-calculator
mvn clean package -DskipTests
# Produces: target/compound-interest-lambda-1.0.0.jar
```

6. Phase 4 — Infrastructure as Code (AWS CDK)

Step 1: Initialize the CDK Project

```
mkdir infra && cd infra
npx cdk init app --language typescript
npm install aws-cdk-lib constructs
```

Step 2: Create the Stacks

The infrastructure is split into **separate stacks** with explicit dependencies:

Stack 1: **NetworkStack** (**network-stack.ts**)

- Creates a **VPC** with public and private subnets across 2 AZs
- NAT Gateways for private subnet internet access
- Exports: `vpc`

Stack 2: **DatabaseStack** (**database-stack.ts**) — *Removed in V3*

- Creates a **DynamoDB** table `CompoundInterestCalculations`
- Partition key: `calculationId` (String)
- `PAY_PER_REQUEST` billing mode
- Exports: `table`

Stack 3: **KafkaStack** (**kafka-stack.ts**)

- Creates an **Amazon MSK** cluster (3 brokers, `kafka.m5.large`)
- Kafka version: 3.5.1
- Security group allowing ports 9092-9098 from VPC CIDR
- TLS + Plaintext encryption in transit
- Unauthenticated client access
- 100 GB EBS storage per broker
- Exports: `mskCluster` , `mskSecurityGroup`

Stack 4: **BackendStack** (**backend-stack.ts**)

V1 (ECS Fargate):

- ECS Cluster + Fargate Service (Java Spring Boot container)
- Application Load Balancer
- Environment variables: MSK bootstrap servers, DynamoDB table, Kafka topic

V2 (HTTP API + Lambda):

- API Gateway HTTP API
- Lambda function (Java 17) as the API handler
- Routes: `POST /api/calculations` , `GET /api/calculations/{id}`

V3 (WebSocket API + Lambda):

- API Gateway **WebSocket API**
- Lambda function (Java 17) as the API handler
- Routes: `$connect` , `$disconnect` , `$default` , `calculate`
- Environment: `MSK_CLUSTER_ARN` , `KAFKA_TOPIC`

- Security group with VPC access for Kafka connectivity
- Exports: `wsApiId` , `wsUrl`

Stack 5: `LambdaStack` (`lambda-stack.ts`)

- Lambda function from `lambda-calculator/target/*.jar`
- MSK event source mapping (topic: `calculation-requests` , batch size: 10)
- IAM permissions:
 - `kafka-cluster:*` for MSK connectivity
 - `ec2:Describe*` , `ec2:CreateNetworkInterface` for VPC ENI management
 - `execute-api:ManageConnections` for WebSocket push (V3)
 - `dynamodb:PutItem` , `dynamodb:UpdateItem` (V1/V2, removed in V3)

Stack 6: `FrontendStack` (`frontend-stack.ts`)

- S3 bucket for static assets
- CloudFront distribution with S3 origin
- `BucketDeployment` to upload `frontend/dist/`
- Auto-invalidation of CloudFront cache on deploy
- V1/V2: Additional `/api/*` behavior proxying to API Gateway
- V3: S3 origin only, WebSocket URL passed as output

Step 3: Wire Stacks Together (`bin/app.ts`)

```
const network = new NetworkStack(app, 'compound-calc-network', { ... });
const kafka = new KafkaStack(app, 'compound-calc-kafka', { vpc: network.vpc });
const backend = new BackendStack(app, 'compound-calc-backend', {
  vpc: network.vpc, mskSecurityGroup: kafka.mskSecurityGroup, ...
});
const lambda = new LambdaStack(app, 'compound-calc-lambda', {
  vpc: network.vpc, mskCluster: kafka.mskCluster,
  wsApiId: backend.wsApiId, ...
});
const frontend = new FrontendStack(app, 'compound-calc-frontend', {
  wsUrl: backend.wsUrl, ...
});
```

Dependency order: Network → Kafka → Backend → Lambda → Frontend

7. Phase 5 — Local Development with Docker Compose

`docker-compose.local.yml`

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    ports: ["2181:2181"]

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    ports: ["9092:9092"]
    depends_on: [zookeeper]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

  dynamodb-local:
    image: amazon/dynamodb-local:latest
    ports: ["8000:8000"]
```

Local Development Flow

1. Start Docker containers: `docker compose -f docker-compose.local.yml up -d`
2. Run Spring Boot: `cd backend && mvn spring-boot:run -Dspring-boot.run.profiles=local`
3. Run React dev server: `cd frontend && npm run dev`
4. Open `http://localhost:5173`

8. Phase 6 — Build & Deploy to AWS

Step 1: Build All Artifacts

```
# Lambda Calculator JAR
cd lambda-calculator && mvn clean package -DskipTests

# API Lambda JAR (V2+)
cd lambda-api && mvn clean package -DskipTests

# Frontend bundle
cd frontend && VITE_WS_URL=wss://<api-id>.execute-api.us-east-1.amazonaws.com/prod \
  npm run build

# CDK dependencies
cd infra && npm ci
```

Step 2: Bootstrap CDK (First Time Only)

```
cd infra
npx cdk bootstrap aws://<account-id>/us-east-1
```

Step 3: Deploy All Stacks

```
cd infra
AWS_REGION=us-east-1 npx cdk deploy --all --require-approval never \
  --context appName=compound-calc
```

Deployment order (CDK resolves automatically):

1. `compound-calc-network` (~2 min) — VPC, subnets, NAT gateways
2. `compound-calc-kafka` (~20 min) — MSK cluster provisioning
3. `compound-calc-backend` (~2 min) — API Gateway + Lambda
4. `compound-calc-lambda` (~5 min) — Calculator Lambda + MSK event source mapping
5. `compound-calc-frontend` (~2 min) — S3 upload + CloudFront distribution

Total first deploy: ~30-35 minutes (MSK is the bottleneck).

Step 4: Note the Outputs

After deployment, CDK outputs:

```
compound-calc-frontend.CloudFrontUrl = https://d1wydi3m5vw0w9.cloudfront.net
compound-calc-backend.WebSocketUrl = wss://s9qzrqf9md.execute-api.us-east-1.amazonaws.com/prod
```

9. Phase 7 — Migration: ECS Fargate → API Gateway Lambda

Why Migrate

- ECS Fargate runs **24/7** even with zero traffic (~\$30-50/month minimum)
- Lambda is **pay-per-invocation** — free at low traffic
- Eliminates Docker image management, ECR, ALB

Step 1: Create `lambda-api/` Project

New Maven project with:

- `aws-lambda-java-core`
- `kafka-clients`
- `aws-sdk-java` (MSK client for resolving bootstrap brokers)
- `jackson-databind`

Step 2: Write `ApiHandler.java`

The handler:

1. Resolves MSK bootstrap brokers from `MSK_CLUSTER_ARN` at cold start
2. Creates a `KafkaProducer` (reused across warm invocations)
3. Handles `POST /api/calculations` → validate, save to DynamoDB, publish to Kafka
4. Handles `GET /api/calculations/{id}` → read from DynamoDB

Step 3: Update CDK `backend-stack.ts`

Replace ECS Fargate resources with:

- API Gateway HTTP API
- Lambda function (Java 17, 1024 MB memory, 30s timeout)
- Routes proxying to Lambda

Step 4: Rebuild & Redeploy

```
cd lambda-api && mvn clean package -DskipTests
cd infra && npx cdk deploy compound-calc-backend --require-approval never
```

10. Phase 8 — Migration: DynamoDB Polling → WebSocket Push

Why Migrate

- Polling wastes 75-85% of API Lambda invocations (most polls find PENDING)
- 1-second polling interval adds perceived latency
- DynamoDB serves as unnecessary intermediary for transient data

Step 1: Update `CalculationEvent.java`

Add `connectionId` and `wsCallbackUrl` fields so the Calculator Lambda knows where to push results.

Step 2: Rewrite `CompoundInterestHandler.java` (Calculator Lambda)

Before: Compute → DynamoDB PUT **After:** Compute → WebSocket `PostToConnection`

```
ApiGatewayManagementApiClient client = ApiGatewayManagementApiClient.builder()
    .endpointOverride(URI.create(wsCallbackUrl))
    .region(awsRegion)
    .build();

client.postToConnection(PostToConnectionRequest.builder()
    .connectionId(connectionId)
    .data(SdkBytes.fromUtf8String(resultJson))
    .build());
```

Step 3: Update `lambda-calculator/pom.xml`

Replace DynamoDB SDK with API Gateway Management API SDK:

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>apigatewaymanagementapi</artifactId>
</dependency>
```

Step 4: Rewrite `ApiHandler.java` (API Lambda)

Change from HTTP API handler to **WebSocket API** handler:

- Accept raw `Map<String, Object>` input (WebSocket events have different structure)
- Handle routes: `$connect`, `$disconnect`, `calculate`
- Extract `connectionId` and `domainName/stage` from `requestContext`
- Build `wsCallbackUrl` and include in Kafka message
- Remove all DynamoDB reads/writes

Step 5: Update CDK Backend Stack

Replace HTTP API Gateway with **WebSocket API Gateway**:

- Define WebSocket routes: `$connect` , `$disconnect` , `$default` , `calculate`
- Create integrations linking each route to the API Lambda
- Add deployment stage (`prod`)

Step 6: Update CDK Lambda Stack

- Remove DynamoDB table reference and permissions
- Add `execute-api:ManageConnections` permission
- Add EC2 VPC/ENI permissions for MSK event source mapping:
 - `ec2:DescribeSecurityGroups`
 - `ec2:DescribeSubnets`
 - `ec2:DescribeVpcs`
 - `ec2:CreateNetworkInterface`
 - `ec2:DescribeNetworkInterfaces`
 - `ec2:DeleteNetworkInterface`

Step 7: Remove DatabaseStack

Delete `database-stack.ts` and remove from `bin/app.ts` .

Step 8: Update CDK Frontend Stack

- Remove `/api/*` CloudFront proxy behavior
- Pass `wsUrl` instead of `apiUrl`

11. Phase 9 — Frontend Update for WebSocket

Step 1: Rewrite `calculationService.js`

Replace Axios REST + polling with WebSocket client:

```
let socket = null;

export function connect() {
  const url = import.meta.env.VITE_WS_URL;
  socket = new WebSocket(url);
  // Handle onopen, onmessage, onerror, onclose
}

export function sendCalculation(formData) {
  socket.send(JSON.stringify({ action: 'calculate', ...formData }));
}

export function onResult(callback) {
  // Called when message with status: 'COMPLETED' arrives
}
```

Key details:

- Handle wrapped response format `{ statusCode, body }` from API Lambda
- Parse `body` if it's a JSON string
- Route messages based on `status` field: `COMPLETED`, `FAILED`, `PENDING`

Step 2: Update `App.jsx`

- Replace polling state machine with WebSocket lifecycle
- Add `useRef` to store submitted form data (`lastSubmission`)
- Merge form data with WebSocket result in `onResult` callback (ensures `ResultDisplay` has all fields)
- Add WebSocket connection status indicator (Connected/Disconnected)
- Update status messages: `connecting`, `submitting`, `processing`, `done`

Step 3: Build with WebSocket URL

```
cd frontend
VITE_WS_URL=wss://s9qzrqf9md.execute-api.us-east-1.amazonaws.com/prod npm run build
```

The `VITE_WS_URL` is baked into the bundle at build time via Vite's `import.meta.env`.

12. Phase 10 — Deployment Issues & Fixes

Issue 1: API Lambda Handler Input Type

Problem: Used `APIGatewayProxyRequestEvent` which lacks WebSocket-specific fields (`routeKey` , `connectionId`).

Fix: Changed handler signature to `Map<String, Object>` and extracted WebSocket fields manually from `requestContext` .

Issue 2: Kafka Topic Not Found

Problem: Topic `calculation-requests` not present in metadata after `10000 ms` . MSK has `auto-create-topics` disabled.

Fix: Added `AdminClient` in `ApiHandler` constructor to create the topic programmatically if it doesn't exist:

```
AdminClient admin = AdminClient.create(adminProps);
admin.createTopics(Collections.singletonList(
    new NewTopic("calculation-requests", 3, (short) 2)
));
```

Issue 3: Kafka Producer Timeout

Problem: Default `max.block.ms` (60s) exceeded Lambda's 30s timeout, causing silent hangs.

Fix: Added `MAX_BLOCK_MS_CONFIG = 10000` and replaced `flush()` with `Future.get(15, TimeUnit.SECONDS)` for controlled timeout.

Issue 4: MSK Event Source Mapping Disabled

Problem: Calculator Lambda missing EC2 VPC permissions: `ec2:DescribeSecurityGroups` , `ec2:DescribeSubnets` , etc.

Fix: Added all 6 EC2 VPC/ENI permissions to the Lambda role in `lambda-stack.ts` .

Issue 5: Stuck Event Source Mapping

Problem: Event source mapping stuck in `Enabling` state even after fixing permissions.

Fix: Deleted the stuck mapping, destroyed the lambda stack, and redeployed fresh:

```
aws lambda delete-event-source-mapping --uuid <uuid>
npx cdk destroy compound-calc-lambda --force
npx cdk deploy compound-calc-lambda --require-approval never
```

13. Architecture Evolution Summary

V1 → V2 → V3

V1:	Browser → CloudFront → ALB → ECS Fargate → Kafka → Lambda → DynamoDB → (poll) (Spring Boot)
V2:	Browser → CloudFront → API Gateway (HTTP) → Lambda API → Kafka → Lambda → DynamoDB → (poll)
V3:	Browser → CloudFront (static only) Browser → WebSocket API Gateway → Lambda API → Kafka → Lambda → WebSocket push

What Was Removed at Each Stage

Migration	Removed
V1 → V2	ECS Fargate, ALB, ECR, Docker image build
V2 → V3	DynamoDB, DatabaseStack, HTTP API Gateway, REST polling, /api/* CloudFront proxy

Final Service Count

Service	Role
CloudFront + S3	Static frontend hosting
API Gateway (WebSocket)	Bidirectional client communication
Lambda (API)	Request validation, Kafka publishing
Amazon MSK	Event streaming (Kafka)
Lambda (Calculator)	Compound interest computation
VPC	Network isolation

14. Lessons Learned

Infrastructure

1. **MSK is expensive** (~\$490/month for 3 × m5.large). For demo/learning projects, consider MSK Serverless or SQS.
2. **VPC Lambda ENI cleanup** takes 20-40 minutes during stack deletion. Plan for this in CI/CD pipelines.
3. **MSK event source mappings** need EC2 VPC permissions that aren't obvious — `ec2:DescribeSecurityGroups` is required but not documented prominently.
4. **CDK cross-stack exports** create implicit deletion dependencies. Delete leaf stacks first.

Backend

5. **Kafka** `max.block.ms` defaults to 60s — too long for Lambda. Always set explicit timeouts.
6. **MSK auto-create-topics** is disabled by default. Either enable it in the cluster config or create topics programmatically.
7. **WebSocket API Gateway** events use a different request structure than HTTP API — typed SDK classes don't fully support it. Use raw `Map<String, Object>` .

Frontend

8. **VITE_WS_URL** is baked at build time. Changing the WebSocket URL requires a frontend rebuild and redeploy.
9. **Merge form data with WebSocket result** — the server may not echo back all original inputs. Use `useRef` to preserve the submitted form data and merge.

Operations

10. **Parallel stack deletion** is limited by cross-stack export dependencies. The maximum parallelism for this project was deleting `frontend` + `lambda` simultaneously.
11. **Stuck CloudFormation resources** (like event source mappings) sometimes require manual deletion followed by stack destroy/redeploy.
12. **Always check Lambda logs** (`aws logs tail /aws/lambda/<name>`) when debugging invocation issues. Timeout errors and permission issues are only visible there.